Relatório da atividade individual de Pinia

Projeto: CTI Insights
Aluno: _____
RA: _____
Turma: _____
GitHub: https://github.com/recheraphael-Re/CTI_SENAI

O projeto utiliza Vue 3 e Pinia para receber uma planilha, armazenar os registros temporariamente, validar os dados e apresentar um relatório. O fluxo implementado é **planilha → tela de upload → store do Pinia → tela de relatório**. Este documento descreve os cinco pontos solicitados na atividade individual.

1 O que foi armazenado no state

A store está em `FrontEnd/src/stores/uploadStore.js`, definida por `defineStore('upload', ...)`. Foi utilizado o formato Setup Store: `ref` e `shallowRef` representam o state, `computed` representa os getters e as funções representam as actions.

No state foram armazenados o arquivo selecionado (`arquivo`), os dados originais da planilha (`dadosOriginais`), os registros tratados (`clientes`), as ocorrências de erro (`erros`) e o resumo da validação (`resumo`). Cada cliente mantém sua linha na planilha, os campos tratados, a lista de erros e a indicação de registro válido ou inválido.

Também são armazenados o nome do arquivo (`nomeArquivo`), a data do processamento (`dataUpload`), a quantidade de colunas (`totalColunas`), o período do faturamento (`periodoFaturamento`), o estado de carregamento (`carregando`) e a mensagem de falha da operação (`erro`). `erros` reúne problemas dos registros; `erro` informa falhas que impedem a leitura.

Os resultados ficam em memória e são gravados em `sessionStorage` para recuperação após atualizar a página na mesma sessão. O objeto File não é restaurado. A planilha não é enviada a um backend.

2 Quais getters foram criados e o que calculam

- `totalLinhas`, `totalValidas` e `totalComErro`: total de registros importados, quantidade válida e quantidade com erro.
- `linhasValidas` e `linhasComErro`: listas de registros separados pelo resultado da validação.
- `totalErros` e `errosPorTipo`: total de ocorrências e contagem agrupada por tipo de erro.
- `dadosTratados`, `totalClientes` e `clientesNivelA`: acesso à base tratada, quantidade de clientes válidos e quantidade válida de nível A.
- `temDados`, `usandoExemplo` e `clientesExibidos`: indicação de base importada ou demonstrativa e seleção da lista exibida nos indicadores.

- `clientesValidos` e `faturamentoTotal`: lista válida usada nos indicadores e soma de seu faturamento.
- `indicadores` e `faturamentoPorSegmento`: clientes válidos, faturamento médio, registros ativos e soma por segmento dos dez maiores totais.
- `rotuloFaturamento`: texto que identifica o período mensal, anual ou não informado.

Uma linha pode apresentar vários erros; por isso, registros com erro e ocorrências podem ter quantidades diferentes. O relatório utiliza apenas registros importados. Os indicadores podem mostrar uma amostra enquanto nenhuma planilha foi carregada.

3 Quais actions foram implementadas

- `selecionarArquivo(novo)`: verifica a extensão XLSX, XLS ou CSV e guarda o arquivo. Bloqueia uma nova seleção durante o processamento.
- `validarArquivo()`: confere se existe um arquivo selecionado com extensão aceita.
- `processarPlanilha()`: inicia a importação do arquivo selecionado.
- `importarPlanilha(arquivoSelecionado)`: controla o carregamento, executa a leitura e a validação, atualiza o state, salva os resultados na sessão e informa falhas. Retorna `true` em caso de sucesso e `false` em caso de falha ou operação descartada.
- `limpar()`: remove dados e erros, apaga a persistência da sessão e invalida resultados de operações anteriores ainda pendentes.

As regras foram organizadas em `FrontEnd/src/utils/planilha.js`. A action de importação executa esse módulo e armazena os resultados na store. As telas consomem estado e getters com `storeToRefs`, preservando a reatividade.

4 Quais validações foram realizadas

- **Campos obrigatórios:** registra `CAMPO_OBRIGATORIO` para código, cliente, consultor, serviço, segmento, nível e faturamento vazios.
- **Duplicidade:** registra `DUPLICIDADE` a partir da segunda ocorrência do mesmo código de cliente.
- **E-mail:** quando a coluna `email` existe, registra `EMAIL_INVALIDO` para formato inadequado e `CAMPO_OBRIGATORIO` para e-mail vazio. A verificação é de formato, sem comprovar a existência da caixa postal.
- **Dados fora do padrão:** registra `DOMINIO_INVALIDO` para nível ou status preenchido fora das listas de referência. A conferência usa o texto original, mesmo que a apresentação seja padronizada.
- **Números:** registra `NUMERO_INVALIDO` para conteúdo não numérico preenchido e `VALOR_NEGATIVO` para faturamento ou valor de serviço negativo.
- **Datas:** registra `DATA_INVALIDA` para data preenchida que não pode ser interpretada, incluindo datas inexistentes como 31/02/2026.

- **Espaços e textos:** remove espaços nas extremidades, reduz espaços repetidos em cliente e consultor e padroniza grafias de segmento, serviço, nível e status. Por exemplo, `IND.` é apresentado como `Indústria`. Os valores originais são preservados.
- **Estrutura do arquivo:** interrompe a leitura se faltarem colunas obrigatórias, houver cabeçalhos ambíguos ou não forem encontrados registros.

A coluna de e-mail é opcional porque a base oficial não a contém. Quando presente, todos os e-mails da coluna são verificados. O arquivo `Data/Exemplo_Validacao_Email.csv` permite demonstrar esse caso. Espaços e grafias tratáveis são corrigidos para apresentação e não geram, por si só, uma ocorrência de erro. Linhas com erros ficam fora dos indicadores.

5 Como os resultados são apresentados no sistema

Na tela `/app/upload`, o usuário seleciona ou arrasta a planilha e clica em **Enviar e processar**. O sistema mostra o nome do arquivo, a aba importada, as contagens da validação, as ocorrências por tipo e uma prévia dos dez primeiros registros tratados. Falhas de leitura aparecem na própria tela.

Na tela `/app/relatorios`, são apresentados o nome do arquivo, a data do processamento, o total de registros, a quantidade válida, a quantidade com erro e a contagem de cada tipo de erro. A tabela informa a linha de origem e, para cada problema, o campo afetado, o tipo e a descrição. Registros inválidos recebem destaque visual; os demais mostram **Sem erros**.

O relatório permite buscar clientes e filtrar somente linhas com erro. Sem importação, orienta o usuário a enviar uma planilha. O botão **Descartar planilha**, na tela de upload, limpa os resultados.

5.1 Verificação da entrega

A base oficial `Data/CTI_Insights_Base_Validacao_300_Registros.xlsx` produz **300 registros, 265 válidos, 35 com erro e 37 ocorrências**, conferidos pelo teste que compara o resultado com o gabarito da planilha. E-mail e limpeza de espaços possuem um teste específico. Nesta revisão, os oito testes passaram e a compilação foi concluída.

Comandos na pasta `FrontEnd`:

```
npm.cmd install
npm.cmd run dev
node --test --test-isolation=none tests/*.test.js
npm.cmd run build
```

Importando `Data/Exemplo_Validacao_Email.csv`, o resultado esperado é **4 registros, 1 válido e 3 com erro**, sendo duas ocorrências de `EMAIL_INVALIDO` e uma de `CAMPO_OBRIGATORIO`. No registro válido, os espaços excedentes de cliente e consultor são removidos.

Entrega: preencher a identificação deste documento, entregar o Word e informar o link do GitHub acima.